这就是开源

第一讲：从操作系统到开源

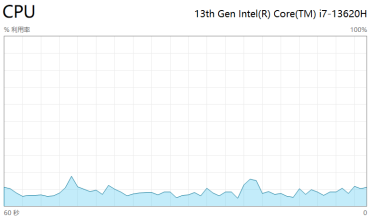
潘宇琦　申鹏威

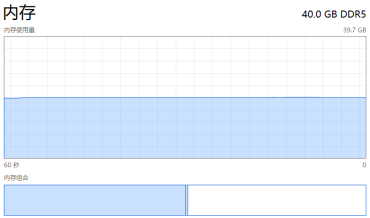
2026 年 10 月

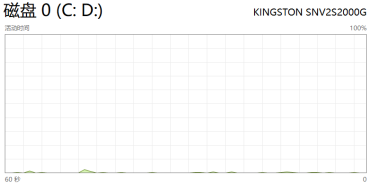
计算机里面有什么？

随便抓来一台电脑

- 处理器 (CPU): 厨师，按照菜谱做菜 (执行程序)
- 内存 (RAM)：案板，临时放置正在处理的食材 (正在运行的程序)
- 硬盘 (Disk)：冰箱，长期保存食材和菜谱 (文件与程序)







为什么我们需要操作系统

如果你正在使用电脑...

这时，你的导员在 QQ 上给你发了文件，你的部长在微信上给你发了通知，你的同学在抖音上给你转发了视频... 那处理器该先给谁用呢？内存该怎么分配呢？

我们既想让它们运行又不想让它们挤破头，该怎么办呢？

我们需要一个管理者！

就像厨房需要厨师长安排厨师的工作、分配厨具的使用一样，计算机也需要一个管理者，负责协调各个程序对硬件资源的使用。

它要安排程序何时使用处理器、合理分配内存，还要尽可能避免不同程序之间的相互干扰。

这个管理者，就是操作系统（Operating System，OS）。

为什么我们需要操作系统

当有人在微信上给你发送了一张图片

当你点击“保存”时，程序需要将图片保存到硬盘上。但程序应该怎样操作硬盘呢？

没有统一的系统服务

程序需要自行处理存储设备的操作细节，甚至适配不同的硬件。

有操作系统提供服务

程序只需要请求“写入文件”，操作系统就可以帮助完成底层操作。

保存图片的过程（简化）

微信 $\longrightarrow$ 操作系统 $\longrightarrow$ 存储设备

操作系统为应用程序提供统一的服务，让程序不必直接处理复杂的硬件细节。

什么是 OS

操作系统（Operating System，OS）

操作系统是管理计算机硬件与软件资源，并为应用程序提供运行环境和基础服务的系统软件。

资源管理者

- 安排处理器工作
- 分配内存空间
- 管理文件与设备

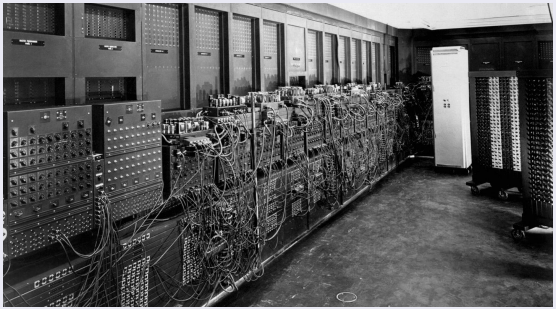
程序服务者

- 帮助程序读写文件
- 提供统一的设备接口
- 为程序提供运行环境

操作系统既是资源的管理者，也是连接应用程序与计算机硬件的桥梁。

操作系统发展史

世界早期的电子计算机——ENIAC



但是

这时的电子计算机尚未拥有现代意义上的操作系统，程序的装载、准备和运行需要大量人工参与，十分复杂。

操作系统发展史

真的需要那么多人工参与吗？

假设我们有 100 个计算任务，如果每完成一个任务，都需要工作人员手动准备下一个任务，不仅麻烦，还会让昂贵的计算机白白等待。

能不能让计算机自己工作呢？

能不能提前把任务准备好，让计算机完成一个任务后，自动开始执行下一个任务呢？

$$\text{任务一} \longrightarrow \text{任务二} \longrightarrow \text{任务三} \longrightarrow \cdots$$

单道批处理系统（Simple Batch System）

20 世纪 50 年代，批处理系统逐渐出现。它能够自动组织并依次执行一批作业，减少人工干预，提高计算机的使用效率，但此时的批处理系统，还是一次只运行一次作业。

操作系统发展史



图: 单道批处理

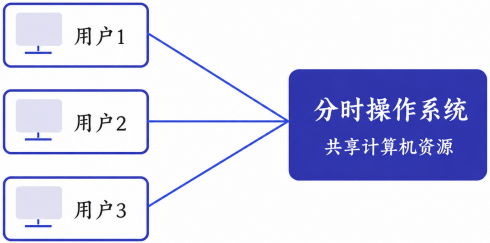


图: 多道批处理

分时操作系统（Time-Sharing System）

批处理系统虽然提高了计算机的使用效率，但用户提交作业后，往往需要等待才能得到结果，难以直接与计算机交互。

20 世纪 60 年代，分时操作系统逐渐发展起来。它通过时间片等调度机制，让多个用户的程序轮流使用处理器，使每个用户都能通过终端与计算机交互，并获得较为及时的响应。



操作系统发展史

话说贝尔实验室

20 世纪 60 年代，MIT、贝尔实验室与通用电气公司合作开发了一套名为 Multics 的分时操作系统。但是，随着系统越来越复杂，开发难度和成本也越来越高，开发进度不及预期，最后贝尔实验室退出了该项目。

能不能简单一些呢？UNIX！

贝尔实验室的研究人员充分吸取了经验教训，不追求一开始就实现大量复杂功能，而是先构建一套小巧、实用、可以不断扩展的系统，也就是后来的 UNIX。

- 多用户：允许多个用户通过终端共享计算机资源
- 简洁性：使用层次化文件系统，通过 Shell 和各种小工具完成复杂任务
- 可移植性：系统不必完全依赖某一种计算机硬件

操作系统发展史

UNIX 走出实验室

凭借优秀的设计与良好的可移植性，UNIX 逐渐从研究机构走向大学与商业市场，吸引了越来越多的开发者和企业参与。

早期的一些大学和研究机构甚至能够获得 UNIX 的源代码，研究它的工作原理，并在许可范围内改进它。

好景不长

为了获得竞争优势，不同厂商纷纷为自己的 UNIX 系统增加独有的功能和接口。这些系统虽然都属于 UNIX 家族，却不一定能兼容彼此的软件。

- 对开发者：同一个程序可能需要针对不同的 UNIX 系统进行反复修改和适配
- 对用户：更换系统可能意味着高昂的迁移成本，以及对特定的厂商的依赖

各大厂商围绕技术标准与商业利益陷入了纷争

从 UNIX 到自由软件

能不能换一种方式？

既然 UNIX 的设计已经如此成熟，我们为什么不能从头开发一套类似的操作系统，不再依赖商业 UNIX 的专有源代码呢？

GNU 计划（GNU Project）

20 世纪 80 年代，Richard Stallman 发起了 GNU 计划，希望开发一套与 UNIX 兼容，但允许用户自由使用、研究、修改和分享的操作系统。

GNU 是“GNU's Not Unix”的递归缩写，意为“GNU 不是 UNIX”。

自由软件（Free Software）

GNU 计划不仅希望开发新的操作系统，更希望建立一种允许用户自由使用软件、研究源代码、分享改进成果的协作方式。

这里的“自由”并不是软件必须免费，而是用户对自己使用的软件享有基本自由。

自由软件

自由软件（Free Software）

自由软件并不是指不收费的软件，而是指能够保障用户使用、研究、修改和分享软件等基本自由的软件。

自由软件的四项基本自由 (具体自由由具体许可证来决定)

- 自由 0：出于任何目的运行软件的自由
- 自由 1：研究软件如何工作，并根据自己的需要修改软件的自由
- 自由 2：复制和再分发软件的自由
- 自由 3：向他人分发修改后版本的自由

自由不等于免费！

假设你开发了一款软件，并将它以 100 元的价格出售，但允许购买者自由使用、研究、修改和分享它。那么，它仍然可以是一款自由软件！

自由软件

自由如何得到保障？

假设我们开发了一款自由软件，允许所有人使用、修改和分享。

但如果有人修改了我们的软件，却不再向后来的软件使用者提供源代码，那么这些用户不就失去了研究和修改它的自由吗？

GNU GPL（GNU General Public License）

为了保障软件用户的自由，GNU 计划推出了 GNU GPL。

GPL 允许用户使用、修改和分享软件，但如果将修改后的 GPL 软件分发给他人，就必须遵守相应的许可条件，使接收者继续享有这些自由，并按规定提供对应的源代码。

Copyleft（著佐权）

Copyleft 利用著作权与软件许可证，使自由软件在被修改和再分发时，仍然能够保持自由。

自由不仅属于最初的用户，也应该属于后续的用户。

从自由软件到开源软件

时代正在发生变化

随着计算机技术的发展，个人计算机逐渐普及，越来越多的普通人拥有了编写软件的条件。与此同时，互联网的发展让世界各地的开发者能够方便地交流代码、分享成果，甚至共同开发同一个软件项目。

开放协作的价值逐渐显现

自由软件的发展让人们逐渐认识到，开放源代码不仅能够保障用户的自由，还能汇聚不同开发者的力量，共同改进软件。1998 年，一部分软件界人士开始推广“开源软件”这一名称，希望强调开放协作的实际价值，吸引更多开发者和企业参与。

两种理念，共同推动软件发展

- 自由软件：更强调用户应当拥有的基本权利
- 开源软件：更强调开放协作带来的实际价值

开源的意义

假如一位学长开发了一款校园软件……

一位学长开发了一款非常实用的课表管理软件，但毕业后没有时间继续维护。后来的同学发现了问题，还希望增加新的功能。难道这个软件就只能停止更新了吗？

如果这是一款开源软件呢？

- 知识共享：其他人可以阅读源代码，学习软件的实现原理
- 协作创新：不同开发者可以共同修复问题、增加功能，并分享改进成果
- 自主维护：即使原作者不再维护，其他人仍有权继续开发这个项目

开源的价值

开源不仅是公开代码，更是通过开放的许可与协作，让更多人有机会参与软件的学习、改进与长期发展。

让创新不止于一个人，让技术能够被更多人共同推动。

开源的意义

技术应该只属于少数人吗？

“在做一数之前，我在很多家机构讲过课，培训机构真的很赚钱，课时费还越来越高，我见到了不少学生，因为高昂的费用，听完试听课就离开了，人生的分岔口就从那里开始放大，我觉得这有点不公平，我不是什么伟人，但我不想看到普通人连努力的机会都没有，所以我辞职建立了一数，高考是公平的，但教育资源是不平等的，一数希望能略微改变这一点，让每个人能相对平等地去竞争。”

开源的意义

开源无法消除所有的不平等，但它能够降低技术的获取门槛，打破部分知识壁垒，让更多人有机会参与技术的发展。

技术进步不应只是少数人的事业，更多人也应该有机会成为创造者。